

Section A: Basic Queries

1. List all films with their titles and release years.

QUERY:

```
SELECT title , release_year FROM film;
```

2. Find the total number of films in the database.

QUERY:

```
SELECT COUNT(*) FROM film;
```

3. Show the names of all customers who rented at least one film.

QUERY:

```
SELECT DISTINCT cus.first_name, cus.last_name  
FROM customer cus  
JOIN rental ren ON cus.customer_id = ren.customer_id;
```

4. Retrieve the titles of films in the "Action" category.

QUERY:

```
SELECT f.title  
FROM film f  
JOIN film_category fc ON f.film_id = fc.film_id  
JOIN category c ON fc.category_id = c.category_id  
WHERE c.name = 'Action';
```

5. Count how many films belong to each category.

QUERY:

```
SELECT c.name AS category_name, COUNT(fc.film_id) AS film_count  
FROM category c  
JOIN film_category fc ON c.category_id = fc.category_id  
GROUP BY c.name;
```

6. Display the top 5 longest films by length.

QUERY: SELECT title, length

```
FROM film
```

```
ORDER BY length DESC
```

```
LIMIT 5;
```

7.Find the number of rentals made in January 2006.

QUERY:

```
SELECT COUNT(*) AS total_rentals
FROM rental
WHERE rental_date >= '2006-01-01'
AND rental_date < '2006-02-01';
```

8.Show all films that have rental rates greater than \$3.

QUERY:

```
SELECT title, rental_rate
FROM film
WHERE rental_rate > 3;
```

9.List all distinct cities where customers live.

QUERY:

```
SELECT DISTINCT ci.city
FROM customer c
JOIN address a ON c.address_id = a.address_id
JOIN city ci ON a.city_id = ci.city_id;
```

10.Find the staff members who processed rentals.

QUERY:

```
SELECT DISTINCT st.first_name, st.last_name
FROM staff st
JOIN rental ren ON st.staff_id = ren.staff_id;
```

Section B: Intermediate Queries

11. Find the top 10 films by number of rentals.

QUERY:

```
SELECT f.title, COUNT(r.rental_id) AS rental_count
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY f.title
ORDER BY rental_count DESC
LIMIT 10;
```

12. Show the revenue generated by each film (sum of payments).

QUERY:

```
SELECT f.title, SUM(p.amount) AS total_revenue
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY f.title
ORDER BY total_revenue DESC;
```

13. List customers who rented more than 20 films.

QUERY:

```
SELECT c.first_name, c.last_name, COUNT(r.rental_id) AS rental_count
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
GROUP BY c.customer_id
HAVING COUNT(r.rental_id) > 20;
```

14.Find the average rental duration for each category.

QUERY:

```
SELECT c.name AS category_name, AVG(f.rental_duration) AS avg_duration
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
GROUP BY c.name;
```

15.Display films that were rented but never returned late.

QUERY:

```
SELECT DISTINCT f.title
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
WHERE r.return_date <= r.rental_date + INTERVAL '1 day' * f.rental_duration;
```

16.Identify the most popular film category by rental count.

QUERY:

```
SELECT c.name, COUNT(r.rental_id) AS rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN inventory i ON fc.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY c.name
ORDER BY rental_count DESC
LIMIT 1;
```

17.Show the monthly rental count trend for 2005–2006.

QUERY:

```
SELECT DATE_TRUNC('month', rental_date) AS month, COUNT(*) AS rental_count
FROM rental
WHERE rental_date BETWEEN '2005-01-01' AND '2006-12-31'
GROUP BY month
ORDER BY month;
```

18. Find customers who rented films from more than 5 categories.

QUERY:

```
SELECT c.first_name, c.last_name, COUNT(DISTINCT cat.category_id) AS category_count
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film_category fc ON i.film_id = fc.film_id
JOIN category cat ON fc.category_id = cat.category_id
GROUP BY c.customer_id
HAVING COUNT(DISTINCT cat.category_id) > 5;
```

19. List films that were rented by customers from Canada.

QUERY:

```
SELECT DISTINCT f.title
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN customer c ON r.customer_id = c.customer_id
JOIN address a ON c.address_id = a.address_id
JOIN city ci ON a.city_id = ci.city_id
JOIN country co ON ci.country_id = co.country_id
WHERE co.country = 'Canada';
```

20. Show the top 5 customers by total payment amount.

QUERY:

```
SELECT c.first_name, c.last_name, SUM(p.amount) AS total_spent
FROM customer c
JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id
ORDER BY total_spent DESC
LIMIT 5;
```

Section C: Advanced Queries

21. Use a window function to rank films by rental frequency.

QUERY:

```
SELECT f.title,  
       COUNT(r.rental_id) AS rental_count,  
       RANK() OVER (ORDER BY COUNT(r.rental_id) DESC) AS rank  
FROM film f  
JOIN inventory i ON f.film_id = i.film_id  
JOIN rental r ON i.inventory_id = r.inventory_id  
GROUP BY f.title;
```

22. Create a CTE to calculate monthly revenue and display the top 3 months.

QUERY:

```
WITH monthly_revenue AS (  
    SELECT DATE_TRUNC('month', payment_date) AS month,  
           SUM(amount) AS revenue  
    FROM payment  
    GROUP BY month  
)  
SELECT *  
FROM monthly_revenue  
ORDER BY revenue DESC  
LIMIT 3;
```

23. Find films that generated revenue above the overall average.

QUERY:

```
SELECT f.title, SUM(p.amount) AS total_revenue
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
JOIN payment p ON r.rental_id = p.rental_id
GROUP BY f.title
HAVING SUM(p.amount) >
(
    SELECT AVG(total_rev)
    FROM (
        SELECT SUM(p2.amount) AS total_rev
        FROM inventory i2
        JOIN rental r2 ON i2.inventory_id = r2.inventory_id
        JOIN payment p2 ON r2.rental_id = p2.rental_id
        GROUP BY i2.film_id
    ) sub
);
```

24. Write a query to calculate customer lifetime value (total payments per customer).

QUERY:

```
SELECT c.first_name, c.last_name, SUM(p.amount) AS lifetime_value
FROM customer c
JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id
ORDER BY lifetime_value DESC;
```

25. Identify films that were rented in consecutive months.

QUERY:

```
SELECT DISTINCT f.title
FROM film f
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r1 ON i.inventory_id = r1.inventory_id
JOIN rental r2 ON i.inventory_id = r2.inventory_id
WHERE DATE_TRUNC('month', r2.rental_date) =
      DATE_TRUNC('month', r1.rental_date) + INTERVAL '1 month';
```

26. Create a materialized view showing film popularity by category.

QUERY:

```
DROP MATERIALIZED VIEW film_popularity_by_category;
CREATE MATERIALIZED VIEW film_popularity_by_category AS
SELECT c.name AS category_name,
      f.title,
      COUNT(r.rental_id) AS rental_count
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
JOIN film f ON fc.film_id = f.film_id
JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY c.name, f.title;
```


27. Write a stored procedure to calculate revenue for a given film ID.

QUERY:

```
CREATE OR REPLACE PROCEDURE calculate_film_revenue(IN input_film_id INT)
LANGUAGE plpgsql
AS $$
DECLARE
    total_revenue NUMERIC := 0;
BEGIN
    SELECT SUM(p.amount)
    INTO total_revenue
    FROM payment p
    JOIN rental r ON p.rental_id = r.rental_id
    JOIN inventory i ON r.inventory_id = i.inventory_id
    WHERE i.film_id = input_film_id;

    RAISE NOTICE 'Total revenue for film ID % is %', input_film_id, total_revenue;
END;
$$;
```

28. Build a trigger that logs whenever a rental is returned late.

QUERY:

```
CREATE TABLE late_return_log (
    log_id SERIAL PRIMARY KEY,
    rental_id INT NOT NULL,
    customer_id INT NOT NULL,
    film_id INT NOT NULL,
    due_date TIMESTAMP NOT NULL,
    return_date TIMESTAMP NOT NULL,
    logged_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

29.Partition rentals by year and calculate yearly revenue.

QUERY:

```
SELECT EXTRACT(YEAR FROM r.rental_date) AS rental_year,  
       SUM(p.amount) AS yearly_revenue  
FROM rental r  
JOIN payment p ON r.rental_id = p.rental_id  
GROUP BY rental_year  
ORDER BY rental_year;
```

30.Optimize a query using indexes to speed up film search by title.

QUERY:

```
SELECT film_id, title, release_year  
FROM film  
WHERE title = 'Inception';
```